

Take-Home Assignment: AI / ML Engineer

Digital Liver, World-Model Team

About this exercise

This is a self-contained modeling problem drawn from our domain. We like it because it has real tensions in it — accuracy, hard constraints, and explainability pulling against each other — so it shows us how you reason, not how fast you can wire up a tutorial.

A few things up front. There is no single right answer, and we do not expect a complete solution. A focused, well-reasoned slice tells us far more than a sprawling attempt at everything. We care about the quality of your thinking, the defensibility of your choices, and the honesty of your evaluation — including the parts that did not work. So scope it down, make calls under ambiguity, and tell us where and why you drew the line.

Budget **roughly 6-8 hours** (a weekend at most). If that forces you to cut, cut — and say what you cut and why. A gentle on-ramp: get a generator producing trajectories, train the simplest model you can to predict the next state, and look at where it goes wrong. From there you can reach toward the harder parts (the constraints, the "did it really learn?" test, explainability) as far as time allows. A thoughtful start on those beats rushing all of them.

The Digital Liver world model

We work inside a digital twin of a patient's liver disease that moves over time, built around one organizing idea:

A single, low-dimensional, bounded clinical state vector $\mathbf{x}(t)$ is the organizing construct. It plays three roles at once: (1) the thing a generator emits data from, (2) the decode target of a learned world model, and (3) an auditable record of "what the model believes about this patient right now."

In this exercise every observable modality (blood panels, the MRCP/ultrasound image, histology, the 3-D liver shape) is treated as a pure function of $\mathbf{x}(t)$, so an error in $\mathbf{x}$ shows up across every modality at once. (*This is a deliberate simplification — our real imaging pipeline has its own stochastic substrate on top of $\mathbf{x}(t)$. For this exercise, the clean version is what you model.*) Cirrhosis is derived from fibrosis by a fixed monotone function and never stored, so it can never disagree with F .

The task: given a patient's trajectory so far, $\mathbf{x}(t_0 \dots t_k)$, predict how the disease moves, $\hat{\mathbf{x}}(t_{k+1} \dots t_n)$. Our team's working direction is a **JEPA-style predictive model** — one that predicts the *representation* (latent) of a future state rather than its raw values — and we are most interested in seeing that route explored. More on why, and on what we expect you to wrestle with inside it, below.

The state vector $\mathbf{x(t)}$

Monthly timesteps, $\mathbf{x(t)} \in \mathbb{R}^8$, every field in $[0,1]$ except M:

idx	field	meaning	range	temporal behaviour
0	F	fibrosis	$[0,1]$	ratchet, non-decreasing
1	D	ductopenia (duct loss)	$[0,1]$	ratchet, irreversible
2	S	biliary strictures	$[0,1]$	ratchet, except may step down at an ERCP event
3	P	portal hypertension	$[0,1]$	ratchet, non-decreasing
4	A	inflammatory activity	$[0,1]$	fast, mean-reverting
5	C	cholestasis	$[0,1]$	fast, with flares
6	M	malignancy hazard accumulator	$[0,2]$	monotone non-decreasing
7	flare	acute cholangitis flare	$[0,1]$	transient, decays

Context constants supplied alongside (not predicted): `disease_class`, `age`, `sex`, `responder` $\in \{0,1\}$, and a treatment timeline (e.g. UDCA start month; ERCP event months).

The constraints — and where they actually get hard

Some fields move only one way: **F, D, P, M are non-decreasing; S is non-increasing except it may drop at an ERCP event**; and the derived cirrhosis stage must stay consistent with F. All fields stay in bounds. These are physically meaningful — a trajectory where fibrosis drifts back down describes something that cannot happen in a real liver — so enforcing them **by construction** rather than hoping the loss learns them is where we want to see your judgment.

Per-field monotonicity has known parameterisations and is the easy part. The interesting part is the **coupling**, because the dynamics do not factor the way a per-field guarantee does: M accumulates as a hazard of *sustained F·C*, flares perturb A and C together and then decay, and treatment suppresses C and A for responders. Enforcing each channel's constraint independently while keeping these interactions faithful is the real tension. If you cannot fully close the gap, tell us why it is hard and what you would try next — that reasoning is exactly what we are reading for.

The synthetic generator — gift and trap

You can produce unlimited labelled trajectories from a seeded dynamical model: ratchets creep upward driven by A and C and a per-patient susceptibility; A mean-reverts but spikes on flares; treatment suppresses C/A for responders; ERCP transiently relieves S; M accumulates as a hazard of sustained F·C. Write your own to this spec, or ask us for a ~60-line reference.

Here is the trap, stated honestly. Within a *single* generator, a model that generalises across held-out patients has, in effect, recovered the generator's update rule — and that is the best any model could do from this data, because here the generator *is* the disease. So "world model vs. generator-inverter" cannot be cleanly settled from inside one generator, and we are not asking you to pretend otherwise. What we *are* asking: design the sharpest generalisation probe you can from this generator (held-out susceptibility, unseen treatment timing, rollouts longer than training), report where the model breaks, and state plainly what your test can and cannot establish. Seeing the ceiling and naming it is worth more to us than a confident claim that ignores it.

The core tradeoff: auditability vs. expressiveness

The heart of the exercise is this: can a predictive model roll $x(t)$ forward without collapsing to a degenerate representation or drifting off the manifold of valid states (broken constraints, modalities that silently disagree), while staying interpretable enough that you can answer "why did the model say this?"

Our recommended route is a JEPA-style predictive model — one that predicts the *representation* (latent) of a future or hidden state rather than its raw values (see the original JEPA write-up if the idea is new to you). It is the direction our team is actively building in, and it is well matched to this problem: it learns dynamics in a latent that need not be the raw clinical state, and it pairs naturally with a graph-attention encoder (attention guided by the disease's causal graph) and a Neural-ODE for continuous-time change. We would like to see this explored seriously.

We are not asking you to take it on faith — the interesting work is *inside* the choice. A learned predictive latent is more expressive than using $x(t)$ directly, but it is freer to violate the constraints, carries a real risk of representation collapse, and is harder to explain. Making a JEPA-style model accurate, on-manifold, *and* auditable on this bounded 8-D state is the genuine challenge, and exactly what we want to watch you reason through. If you weigh it against simpler peers — $x(t)$ as the latent, a plain Neural-ODE, a supervised decoder — and still choose to depart from JEPA, that is allowed, but make the case: departing from our working direction is a higher bar than reasoning well within it, and the strongest contrarian answer engages with what JEPA buys before setting it aside.

What does not land is any architecture bolted on as a buzzword with no account of what it buys you — and that applies to JEPA most of all.

Out of scope (for context only): causal vs. correlational under referral bias

The clinical value of a twin lies in counterfactuals ("what if UDCA had started six months earlier?"), which require respecting the causal graph's interventional semantics, not just attention-weighted correlation — especially when cohorts are referral-stratified and the model is tempted to learn "this patient reached us, therefore they are sick." Validating that against re-runs of the generator is a rich problem on its own, so we keep it out of

scope to keep this exercise focused. We mention it only so you can see how the pieces connect.

Your assignment

Build and evaluate a world model that predicts future liver-disease states that are at once **accurate**, **constraint-respecting** (the one-directional fields must never reverse), and **explainable** enough that a reviewer asking "why this prediction?" gets a real answer. Treat the generator as both gift and quality bar, and design an evaluation that could have proven your own model wrong.

We weight the depth of your reasoning, the honesty of your evaluation, and the judgment calls you make under ambiguity — not polish or completeness.

Deliverables

Weighted toward thinking; code is the evidence, not the point. A focused, honest slice beats an overstretched attempt at all of it.

1. **Decision memo (up to 3 pages) — the primary deliverable.** Why a JEPA-style predictive latent for this problem (and what it buys you over simpler peers — $x(t)$ as the latent, a plain Neural-ODE, a supervised decoder — that you considered); the mechanism that prevents representation collapse in your latent and the metric that would catch it if it crept in; how you keep predicted trajectories on the manifold of valid states and what that costs; where the residual risk is; and what you would do next. (*If you depart from JEPA, this memo is where you make that case.*) **Fold your answer to "why did the model predict decompensation at month 30?" into the memo** for one example trajectory. Write it for a Staff engineer who wants to understand the tradeoffs behind each choice.
2. **A working prototype.** Learns from trajectories, predicts future states, and applies your chosen constraint-enforcement mechanism.
3. **An evaluation harness with honest numbers.** At minimum: predictive accuracy on held-out trajectories, a constraint-violation rate, and the generalisation probe described above — with the cases where it fails shown, not hidden.

(Optional) Anything genuinely new you tried — see below.

What a strong submission demonstrates

- You reason clearly about what a predictive latent buys you here and how you keep it honest, engaging seriously with the JEPA-style route before either committing to it or making a real case to depart.
- You make an explicit, reasoned call on representation collapse — a live risk the moment you use a learned latent — and do something deliberate about it.
- You design an evaluation that could have falsified your model, and you report the failures honestly.

- You treat the constraints as a guarantee to engineer, are explicit about how your mechanism achieves it (constrained parameterisation, projection, or otherwise) and what it costs, and you engage with the *coupling*, not just per-field monotonicity.
- You scope ruthlessly, make a defensible call under ambiguity, and document the wall when you hit one.
- You think about explainability without being told to.

None of this requires exotic machinery. What does not land: an accuracy-only evaluation with no constraint or generalisation metric; a polished demo that hides its failures; or leaving the loss to "learn" the one-directional fields and never checking they hold.

Constraints

No access to our repo or real patient data — synthetic data only; everything you need is here. Stuck on scope? Make a call, write down why, move on. That is part of the exercise.

"Try something new" — seeds, not requirements

A sharp take on the core beats a scattered tour of extras, and none of these are expected:

- A constraint-preserving latent parameterisation so the one-directional fields hold by construction; or an energy-based / regularised approach to guard against collapse.
- Using the causal graph as an attention mask so information flows along biological edges, and testing whether it reduces a referral-bias shortcut.
- A learned "is-this-state-on-the-manifold" critic that scores predicted trajectories.
- A counterfactual rollout ("UDCA started 6 months earlier") validated against a re-run of the generator — a toe into the out-of-scope dimension.

Tell us why your idea might be wrong; we enjoy that as much as the idea.

Submission

Send a repo or zip (prototype, harness, memo). We will schedule a relaxed readout where you walk us through your decisions and we talk through the forks together — the conversation matters as much as the artifact.